

PROJECT REPORT

-Gyan pustak

TEAM DETAILS:

B LOKESH	23CS01012
M PRABHAASH	23CS01044
P PARDHIV	23CS01047
V BHARATH	23CS01065

CONTEXT:

1. ABOUT DATABASE
2. ER DIAGRAM
3. RELATIONAL SCHEMAS
4. FEATURES AND THEIR WORKING
5. CONTRIBUTION

INTRODUCTION:

The **Gyan Pustak** project is an online book management system designed to simplify the process of browsing, selecting, and ordering books for students, while enabling administrators to efficiently manage inventory and track orders.

This project is divided into three main sections:

- I. Database (Using mysql)
- II. Backend (Using mysql connector and python)
- III. Frontend (Using streamlit)

ABOUT DATABASE:

The **Gyan Pustak** system uses a relational database implemented in MySQL to efficiently manage and organize data related to books, students, orders, tickets and administrative operations.

The database consists of **27 tables**, each designed to handle a specific part of the system. These tables are interconnected using primary and foreign keys to maintain data integrity and support complex queries.

```
mysql> show tables;
+-----+
| Tables_in_Gyanpustak |
+-----+
| administrator        |
| author               |
| book_author          |
| book_course          |
| book_keyword         |
| book_subcategory     |
| books                |
| cart                 |
| cart_book            |
| category             |
| course               |
| course_department    |
| course_instructor    |
| customer_support     |
| department           |
| emp_ticket_history   |
| employee             |
| instructor           |
| keyword              |
| orders               |
| orders_book          |
| review              |
| student              |
| subcategory          |
| ticket_history       |
| trouble_ticket       |
| university           |
+-----+
27 rows in set (0.00 sec)
```

1. User Management Tables

These tables store information about different users of the system:

- **student** – Stores student details such as name, email, department, and login credentials.
- **administrator** – Contains admin details who manage the system.
- **employee** – Stores employee data for handling support and operations.

2. Book Management Tables

These tables manage book-related data:

- **books** – Main table containing book details such as ISBN, title, price, quantity, publisher, etc.
- **author** – Stores author information.
- **book_author** – Resolves many-to-many relationship between books and authors.
- **category** – Stores main book categories.
- **subcategory** – Stores subcategories of books.
- **book_subcategory** – Maps books to subcategories.
- **keyword** – Stores keywords related to books.
- **book_keyword** – Links books with keywords for search functionality.

3. Academic Mapping Tables

These tables connect books with academic structures:

- **course** – Stores course details.
- **department** – Contains department information.
- **university** – Stores university data.
- **instructor** – Stores instructor details.
- **book_course** – Links books to courses.
- **course_department** – Maps courses to departments.
- **course_instructor** – Maps instructors to courses.

4. Cart and Order Management

These tables handle the ordering system:

- **cart** – Stores cart details for each student.
- **cart_book** – Maps books added to a cart (many-to-many).

- **orders** – Stores order details such as student ID, order date, and status.
- **orders_book** – Maps books to orders.

5. Feedback and Support System

These tables manage user feedback and issue resolution:

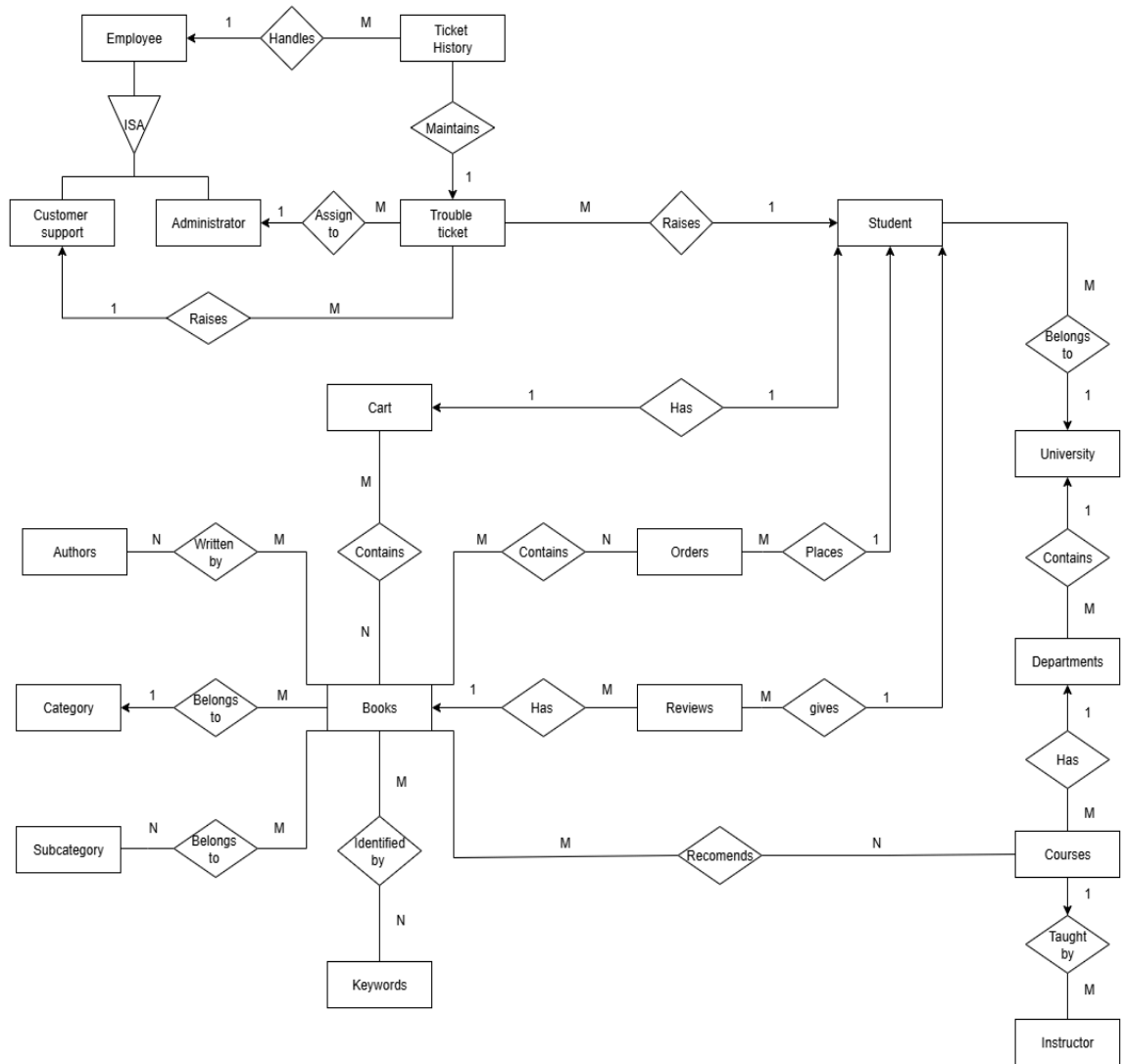
- **review** – Stores student reviews for books.
- **trouble_ticket** – Records issues raised by users.
- **ticket_history** – Tracks updates on tickets.
- **emp_ticket_his** – Maintains employee actions on tickets.
- **customer_support** – Handles support-related data.

ER DIAGRAM:

The Entity-Relationship (ER) diagram of the Gyan Pustak system represents the overall structure and relationships between various entities involved in the application. It models key components such as students, books, cart, orders, and administrative modules.

The diagram highlights how a student can browse books, add them to a cart, and place orders, while also interacting with features like reviews and support tickets. It also captures academic relationships such as courses, departments, and universities, along with book categorization using authors, categories, subcategories, and keywords.

Additionally, the ER diagram includes the support system where trouble tickets are raised, assigned, and maintained by administrators and employees. Overall, it provides a clear visualization of data flow and relationships, forming the foundation for the database design.



RELATIONAL SCHEMA

1) ADMINISTRATOR

```
mysql> desc administrator;
```

Field	Type	Null	Key	Default	Extra
emp_id	int	NO	PRI	NULL	
firstname	varchar(20)	YES		NULL	
lastname	varchar(20)	YES		NULL	
gender	varchar(10)	YES		NULL	
salary	decimal(10,2)	YES		NULL	
aadhaar	varchar(13)	YES		NULL	
email	varchar(50)	YES		NULL	
address	varchar(50)	YES		NULL	
password	varchar(30)	YES		NULL	
phoneno	varchar(11)	YES		NULL	

administrator – Stores details of system administrators who manage operations.

2) AUTHOR

```
mysql> desc author;
```

Field	Type	Null	Key	Default	Extra
author_id	int	NO	PRI	NULL	auto_increment
author_name	varchar(30)	YES		NULL	

author – Contains information about book authors.

3) BOOK_AUTHOR

```
mysql> desc book_author;
```

Field	Type	Null	Key	Default	Extra
author_id	int	NO	PRI	NULL	
isbn	varchar(15)	NO	PRI	NULL	

book_author – Maps books to their authors (many-to-many relationship).

4) BOOK_COURSE

```
mysql> desc book_course;
```

Field	Type	Null	Key	Default	Extra
isbn	varchar(15)	NO	PRI	NULL	
course_id	int	NO	PRI	NULL	

book_course – Links books to relevant academic courses.

5) BOOK_KEYWORD

```
mysql> desc book_keyword;
```

Field	Type	Null	Key	Default	Extra
isbn	varchar(15)	NO	PRI	NULL	
keyword_id	int	NO	PRI	NULL	

book_keyword – Associates books with keywords for search functionality.

6) BOOK_SUBCATEGORY

```
mysql> desc book_subcategory;
```

Field	Type	Null	Key	Default	Extra
subcategory_id	int	NO	PRI	NULL	
isbn	varchar(15)	NO	PRI	NULL	

book_subcategory – Maps books to their respective subcategories.

7) BOOKS

```
mysql> desc books;
```

Field	Type	Null	Key	Default	Extra
isbn	varchar(15)	NO	PRI	NULL	
type	varchar(10)	YES		NULL	
options	varchar(10)	YES		NULL	
price	decimal(10,2)	YES		NULL	
quantity	int	YES		NULL	
title	varchar(40)	YES		NULL	
publisher	varchar(50)	YES		NULL	
publication_date	date	YES		NULL	
edition_no	int	YES		NULL	
language	varchar(30)	YES		NULL	
format	varchar(20)	YES		NULL	
category_id	int	YES	MUL	NULL	

books – Stores complete details of books such as ISBN, title, price, and quantity.

8) CART

```
mysql> desc cart;
```

Field	Type	Null	Key	Default	Extra
cart_id	int	NO	PRI	NULL	auto_increment
date_created	date	YES		NULL	
date_last_updated	date	YES		NULL	
created_by	int	YES	MUL	NULL	

cart – Represents a student's shopping cart.

9) CART_BOOK

```
mysql> desc cart_book;
```

Field	Type	Null	Key	Default	Extra
cart_id	int	NO	PRI	NULL	
isbn	varchar(15)	NO	PRI	NULL	

cart_book – Stores books added to a cart (many-to-many relationship).

10) CATEGORY

```
mysql> desc category;
```

Field	Type	Null	Key	Default	Extra
category_id	int	NO	PRI	NULL	
category_name	varchar(50)	YES		NULL	

category – Contains main categories of books.

11) COURSE

```
mysql> desc course;
```

Field	Type	Null	Key	Default	Extra
course_id	int	NO	PRI	NULL	
course_name	varchar(30)	YES		NULL	
cur_year	int	YES		NULL	
cur_sem	int	YES		NULL	

course – Stores details of academic courses.

12) COURSE_DEPARTMENT

```
mysql> desc course_department;
```

Field	Type	Null	Key	Default	Extra
course_id	int	NO	PRI	NULL	
dept_id	int	NO	PRI	NULL	

course_department – Maps courses to departments.

13) COURSE_INSTRUCTOR

```
mysql> desc course_instructor;
```

Field	Type	Null	Key	Default	Extra
course_id	int	NO	PRI	NULL	
instr_id	int	NO	PRI	NULL	

course_instructor – Links courses with instructors.

14) CUSTOMER_SUPPORT

```
mysql> desc customer_support;
```

Field	Type	Null	Key	Default	Extra
emp_id	int	NO	PRI	NULL	
firstname	varchar(20)	YES		NULL	
lastname	varchar(20)	YES		NULL	
gender	varchar(10)	YES		NULL	
salary	decimal(10,2)	YES		NULL	
aadhaar	varchar(13)	YES		NULL	
email	varchar(50)	YES		NULL	
password	varchar(30)	YES		NULL	
address	varchar(50)	YES		NULL	
phoneno	varchar(11)	YES		NULL	

customer_support – Stores customer support-related information.

15) DEPARTMENT

```
mysql> desc department;
```

Field	Type	Null	Key	Default	Extra
dept_id	int	NO	PRI	NULL	
university_id	int	NO	PRI	NULL	
dept_name	varchar(30)	YES		NULL	

department – Contains details of departments within a university.

16) EMP_TICKET_HIS

```
mysql> desc emp_ticket_his;
```

Field	Type	Null	Key	Default	Extra
change_id	int	NO	PRI	NULL	
emp_id	int	YES	MUL	NULL	

emp_ticket_his – Tracks employee actions on support tickets.

17) EMPLOYEE

```
mysql> desc employee;
```

Field	Type	Null	Key	Default	Extra
emp_id	int	NO	PRI	NULL	auto_increment
firstname	varchar(20)	YES		NULL	
lastname	varchar(20)	YES		NULL	
gender	varchar(10)	YES		NULL	
salary	decimal(10,2)	YES		NULL	
aadhaar	varchar(15)	YES		NULL	
email	varchar(50)	YES		NULL	
phoneno	varchar(11)	YES		NULL	
role	varchar(50)	YES		NULL	
password	varchar(30)	YES		NULL	

employee – Stores details of employees handling support and operations.

18) INSTRUCTOR

```
mysql> desc instructor;
```

Field	Type	Null	Key	Default	Extra
instr_id	int	NO	PRI	NULL	
firstname	varchar(30)	YES		NULL	
lastname	varchar(30)	YES		NULL	
university_id	int	YES	MUL	NULL	
dept_id	int	YES	MUL	NULL	

instructor – Contains instructor details.

19) KEYWORD

```
mysql> desc keyword;
```

Field	Type	Null	Key	Default	Extra
keyword_id	int	NO	PRI	NULL	
keyword	varchar(30)	YES		NULL	

keyword – Stores keywords used for book identification and search.

20) ORDERS

```
mysql> desc orders;
```

Field	Type	Null	Key	Default	Extra
orders_id	int	NO	PRI	NULL	
student_id	int	YES	MUL	NULL	
date_created	date	YES		NULL	
date_fulfilled	date	YES		NULL	
shipping_type	varchar(30)	YES		NULL	
credit_card_number	varchar(20)	YES		NULL	
credit_card_expiry_date	date	YES		NULL	
credit_card_holder_name	varchar(40)	YES		NULL	
order_status	varchar(20)	YES		NULL	

orders – Stores order details placed by students.

21) ORDERS_BOOK

```
mysql> desc orders_book;
```

Field	Type	Null	Key	Default	Extra
orders_id	int	NO	PRI	NULL	
isbn	varchar(15)	NO	PRI	NULL	

orders_book – Maps books to orders (many-to-many relationship).

22) REVIEW

```
mysql> desc review;
```

Field	Type	Null	Key	Default	Extra
review_id	int	NO	PRI	NULL	
isbn	varchar(15)	YES	MUL	NULL	
student_id	int	YES	MUL	NULL	
rating	int	YES		NULL	
description	varchar(100)	YES		NULL	

review – Stores student reviews and ratings for books.

23) STUDENT

```
mysql> desc student;
```

Field	Type	Null	Key	Default	Extra
email	varchar(50)	NO	UNI	NULL	
firstname	varchar(20)	YES		NULL	
lastname	varchar(20)	YES		NULL	
address	varchar(50)	YES		NULL	
phoneno	varchar(11)	YES		NULL	
dob	date	YES		NULL	
university_id	int	YES	MUL	NULL	
dept_id	int	YES	MUL	NULL	
status	varchar(20)	YES		NULL	
curyear	int	YES		NULL	
password	varchar(30)	YES		NULL	
student_id	int	NO	PRI	NULL	auto_increment

student – Contains student information including login and academic details.

24) SUBCATEGORY

```
mysql> desc subcategory;
```

Field	Type	Null	Key	Default	Extra
subcategory_id	int	NO	PRI	NULL	auto_increment
subcategory_name	varchar(40)	YES		NULL	

subcategory – Stores subcategories under main categories.

25) TICKET_HISTORY

```
mysql> desc ticket_history;
```

Field	Type	Null	Key	Default	Extra
change_id	int	NO	PRI	NULL	auto_increment
ticket_id	int	YES	MUL	NULL	
change_date	date	YES		NULL	

ticket_history – Maintains history of updates on support tickets.

26) TROUBLE_TICKET

```
mysql> desc trouble_ticket;
```

Field	Type	Null	Key	Default	Extra
ticket_id	int	NO	PRI	NULL	auto_increment
date_logged	date	YES		NULL	
createdby	varchar(30)	YES		NULL	
title	varchar(20)	YES		NULL	
problem_description	varchar(100)	YES		NULL	
solution_description	varchar(100)	YES		NULL	
completion_date	date	YES		NULL	
emp_id	int	YES	MUL	NULL	
status	varchar(20)	YES		NULL	
student_id	int	YES	MUL	NULL	
support_id	int	YES	MUL	NULL	

trouble_ticket – Records issues raised by students or users.

27) UNIVERSITY

```
mysql> desc university;
```

Field	Type	Null	Key	Default	Extra
university_id	int	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
address	varchar(50)	YES		NULL	
rep_first_name	varchar(20)	YES		NULL	
rep_last_name	varchar(20)	YES		NULL	
email	varchar(50)	YES		NULL	
phoneno	varchar(11)	YES		NULL	

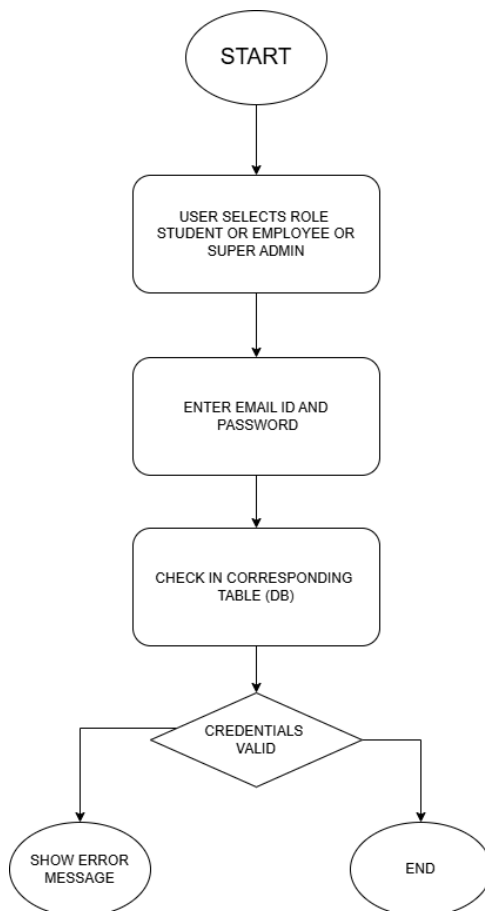
university – Stores university details.

FEATURES AND THEIR WORKING

1. AUTHORIZATION:

This workflow represents the login and access control mechanism of the system. Users select their role such as Student, Customer Support, Administrator, or Super Admin and provide their credentials. The system verifies these details against the respective database tables. If the credentials are valid, the user is granted access and redirected to their role-specific dashboard. If invalid, an error message is displayed. This ensures secure and role-based access to the system.

Work flow :



Backend:

```
if "role" in user:
    role = user["role"]
else:
    role = "student"

if role == "super_admin":
    super_admin_dashboard(user)
elif role == "student":
    student_dashboard(user)
elif role == "customer_support":
    customer_support_dashboard(user)
elif role == "administrator":
    administrator_dashboard(user)
```

Checks corresponding database depending upon which role you have selected.

```
# — USERS —————
def get_superadmin_by_email(email):
    return run_query(
        "SELECT * FROM employee WHERE email=%s and role='super_admin'",
        (email,), fetch=True, one=True
    )

def get_admin_by_email(email):
    return run_query(
        "SELECT * FROM administrator WHERE email=%s",
        (email,), fetch=True, one=True
    )

def get_student_by_email(email):
    return run_query(
        "SELECT * FROM student WHERE email=%s",
        (email,), fetch=True, one=True
    )
```

2.TROUBLE TICKET:

This workflow shows how issues are raised and resolved in the system. A ticket can be created by either a student or customer support, and it is initially marked as “New.” All new tickets are directed to the customer support team for review. After analyzing the issue, customer support assigns the ticket to an administrator. The administrator works on resolving the issue and updates the ticket status. This structured process ensures proper handling and tracking of user issues.

Backend:

```
#ticket by student
def create_ticket_st(student_id, subject, description):
    today = date.today()

    run_query("""
        INSERT INTO trouble_ticket(date_logged, createdby, title, problem_description,
        status, student_id, support_id)
        VALUES (%s,%s,%s,%s,%s,%s,%s)""" , (
        today,"student",subject,
        description,"New",student_id,None
        ))

    ticket_id = run_query("SELECT MAX(ticket_id) as id from trouble_ticket", fetch=True, one=True)["id"]
    # history
    run_query("""
        INSERT INTO ticket_history (ticket_id, change_date)
        VALUES (%s,%s)
        """, (ticket_id, today))
```

```
#ticket by customer support
def create_ticket_cs(support_id, title, description):
    today = date.today()

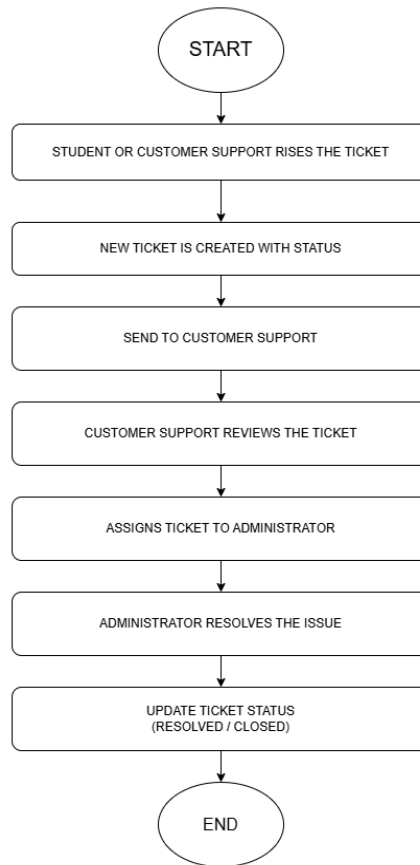
    run_query("""INSERT INTO trouble_ticket(date_logged, createdby, title, problem_description, status, student_id, support_id)
    VALUES (%s,%s,%s,%s,%s,%s,%s)""" , (
    today,"customersupport",title,
    description,"New",None,support_id))

    ticket_id = run_query("SELECT MAX(ticket_id) as id from trouble_ticket", fetch=True, one=True)["id"]
    # history
    run_query("""
        INSERT INTO ticket_history (ticket_id, change_date)
        VALUES (%s,%s)
        """, (ticket_id, today))
```

Ticket gets added to database depending on the role.

```
def assign_ticket(ticket_id, emp_id):
    run_query("""
        UPDATE trouble_ticket
        SET emp_id=%s,
        status='Assigned'
        WHERE ticket_id=%s
        """, (emp_id, ticket_id))
```

Work Flow:



3.ORDER PLACEMENT:

This workflow explains how a student places an order. The student first browses books and adds desired items to the cart. After reviewing the cart, the user proceeds to place an order by filling in required details such as shipping and payment information. The system then creates a new order entry and stores the selected books in the order mapping table. Finally, the cart is cleared and the order status is set to “Placed.” This ensures a smooth and complete transaction process.

Backend:

```

# --- ORDERS ---
def checkout_cart(student_id):
    cart = run_query(
        "SELECT cart_id FROM cart WHERE created_by=%s",(student_id,),fetch=True
    )

    if not cart:
        return False, "Cart not found"

    cart_id = cart[0]["cart_id"]

    items = run_query(
        "SELECT isbn FROM cart_book WHERE cart_id=%s",
        (cart_id,),
        fetch=True
    )

    if not items:
        return False, "Cart empty"

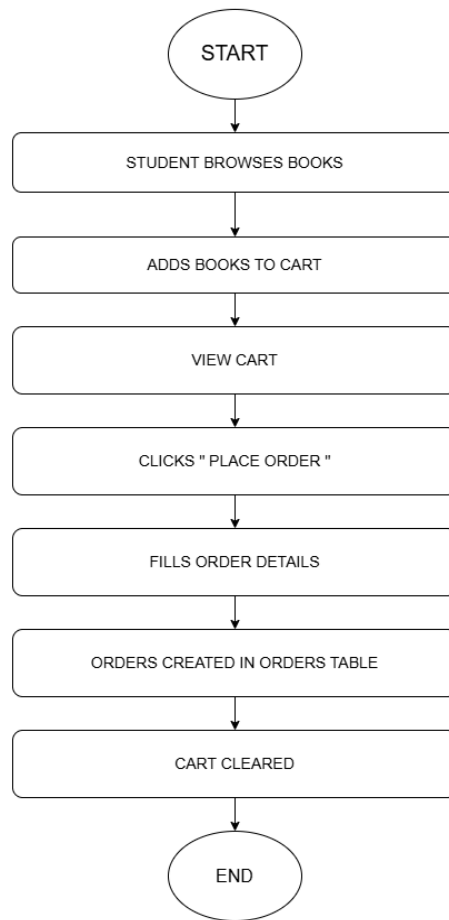
    for item in items:
        run_query(
            "INSERT INTO orders (student_id, isbn, status) VALUES (%s, %s, %s)",
            (student_id, item["isbn"], "pending")
        )

    run_query(
        "DELETE FROM cart_book WHERE cart_id=%s",
        (cart_id,)
    )

    return True, "Order placed"

```

Work Flow:



4.ADD EMPLOYEE:

This workflow describes how new employees are added to the system. Only the Super Admin has permission to perform this action, ensuring centralized control. The Super Admin enters employee details and selects the appropriate role, such as Administrator or Customer Support. The system then stores this information in the database. Once added, the employee can log in and perform their assigned duties. This maintains a controlled and secure employee management system.

Backend:

```
def create_employee(firstname, lastname, gender, salary, aadhaar, email, address, phoneno, role, password):
    try:
        # Step 1: Insert into employee
        run_query(
            """INSERT INTO employee
            (firstname, lastname, gender, salary, aadhaar, email, phoneno, role, password)
            VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)"""
            (firstname, lastname, gender, salary, aadhaar, email, phoneno, role, password)
        )
        emp_id = run_query("select emp_id from employee where email=%s", (email,), fetch=True, one=True)["emp_id"]

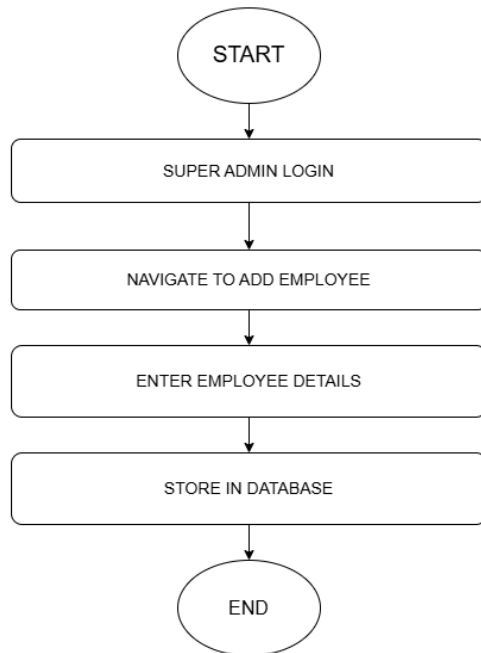
        # Step 2: Insert into role-specific table using SAME emp_id
        if role == "administrator":
            run_query(
                """INSERT INTO administrator
                (emp_id, firstname, lastname, gender, salary, aadhaar, email, address, password, phoneno)
                VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)"""
                (emp_id, firstname, lastname, gender, salary, aadhaar, email, address, password, phoneno)
            )

        elif role == "customer_support":
            run_query(
                """INSERT INTO customer_support
                (emp_id, firstname, lastname, gender, salary, aadhaar, email, password, address, phoneno)
                VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)"""
                (emp_id, firstname, lastname, gender, salary, aadhaar, email, password, address, phoneno)
            )

        return True, "Employee created"

    except Exception as e:
        return False, str(e)
```

Work Flow:



5.ADD BOOKS AND CATEGORIES:

This workflow shows how administrators manage books and categories. After logging in, the administrator can choose to add a new category or a new book. For categories, relevant details are entered and saved in the database. For books, the administrator provides complete information and assigns appropriate categories or subcategories. This ensures proper classification and organization of books. It also helps users easily browse and search for books.

Backend:

If subcategories are not present in the database that will also get added in this section only.

```

#add new book
def add_book(title, author, isbn, category_id, copies,publisher, publication_date, edition_no,language, format_, type_, option, price, subcat_ids):
    try:
        # check duplicate
        if run_query("SELECT 1 FROM books WHERE isbn=%s",
            (isbn,), fetch=True, one=True):
            return False, "ISBN exists"

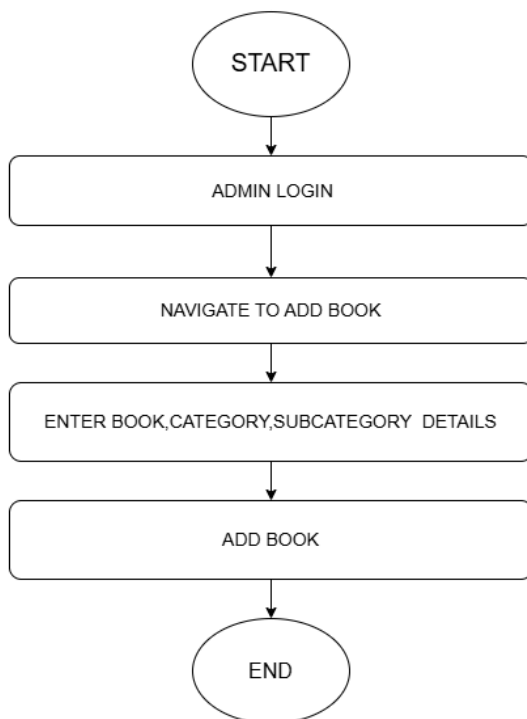
        # insert book
        run_query("""
            INSERT INTO books
            (isbn, title, quantity, category_id, publisher,
            publication_date, edition_no, language,
            format, type, options, price)
            VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)
        """, (
            isbn, title, copies, category_id,
            publisher, publication_date, edition_no,
            language, format_, type_, option, price
        ))

        # author handling
        row = run_query(
            "SELECT author_id FROM author WHERE author_name=%s",
            (author,), fetch=True, one=True
        )

        if row:
            author_id = row["author_id"]
        else:
            run_query(
                "INSERT INTO author (author_name) VALUES (%s)",
                (author,),
                fetch=True )
            author_id = run_query("SELECT MAX(author_id) as id FROM author", fetch=True, one=True)["id"]
        run_query(
            "INSERT INTO book_author (isbn, author_id) VALUES (%s,%s)",
            (isbn, author_id)
        )
    )

```

Work Flow:



6.SUPER ADMIN SUMMARY WORKFLOW:

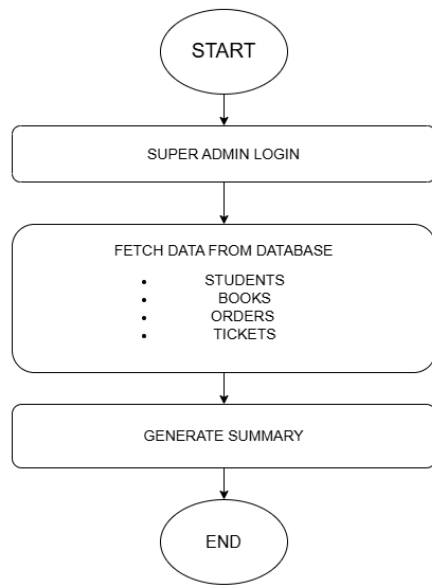
This workflow represents the overall system monitoring process. The Super Admin logs in and retrieves data related to students, books, orders, and tickets from the database. The system processes this data to generate a summary or analytics view. The results are displayed on a dashboard, providing insights into system performance and activity. This helps in effective decision-making and overall system management.

Backend:

All the summary about the management system will be monitored by super admin.

```
# --- STATS ---
def get_dashboard_stats():
    stats = {}
    stats["total_books"] = run_query("SELECT COUNT(*) as c FROM books", fetch=True, one=True)["c"]
    stats["total_students"] = run_query("SELECT COUNT(*) as c FROM student", fetch=True, one=True)["c"]
    stats["total_orders"] = run_query("SELECT COUNT(*) as c FROM orders", fetch=True, one=True)["c"]
    stats["total_employees"] = run_query("SELECT COUNT(*) as c FROM employee", fetch=True, one=True)["c"]
    stats["active_orders"] = run_query("SELECT COUNT(*) as c FROM orders WHERE order_status!='cancelled'", fetch=True, one=True)["c"]
    stats["pending_orders"] = run_query("SELECT COUNT(*) as c FROM orders WHERE order_status='pending'", fetch=True, one=True)["c"]
    stats["open_tickets"] = run_query("SELECT COUNT(*) as c FROM trouble_ticket WHERE status='New'", fetch=True, one=True)["c"]
    return stats
```

Work Flow:



TECH STACK USED:

Frontend

- **Streamlit (Python Framework)**
Used to build an interactive and user-friendly web interface.
- **HTML & CSS (via Streamlit Markdown)**
Used for designing layouts, styling components, and improving UI appearance.

Backend

- **Python**
Core programming language used to implement application logic.
- **MySQL Connector (mysql-connector-python)**
Used to connect Python with the MySQL database and execute queries.

- **Modular Python Scripts**

Separate files handle database operations, user functions, and business logic.

Database

- **MySQL**

Relational database used to store and manage all application data.

- **Structured Tables (27 tables)**

Includes entities like students, books, cart, orders, tickets, etc.

- **SQL Queries**

Used for data retrieval, insertion, updates, and deletion.

Libraries and Dependencies

- `streamlit` – For frontend UI
- `mysql-connector-python` – For database connectivity
- `pandas` (optional) – For data handling and display

Architecture Overview

- **Frontend (Streamlit UI)** interacts with
- **Backend (Python logic)** which communicates with
- **Database (MySQL)**

CONTRIBUTION:

1. Lokesh

.Backend

.Frontend

.Database

2. Pardhiv

.Frontend

- .Report
- .ER diagram

3.Prabhaash

- .Database
- .ER diagram
- .Report

4.Bharath

- .Frontend
- .Report
- .Database